

MÓDULO 5: BACKEND AGÉNTICO

# DEL FRONTEND A LA BASE DE DATOS

Extracción de Contratos, Supabase MCP y Endpoints en Next.js — sin pasos manuales

# El arsenal que traemos de la Sesión 4



## **src/lib/types.ts**

Interfaces TypeScript de User, Ticket, Project, Tag... El contrato de datos ya existe.



## **src/lib/mock-data.ts**

Datos semilla con la forma exacta que el frontend consume. El backend debe responder con esa misma forma.



## **src/store/**

Stores de Zustand con cada acción del usuario: moveTicket, archiveProject, addComment... Cada acción = un endpoint.



## **Specs-MiniJira/backlog.md**

16 historias en Gherkin con criterios de aceptación ejecutables. Claude los usará para validar que los endpoints cubren todos los casos borde.

# Plan de acción:

## **01** **Análisis del Frontend → Contrato de Endpoints**

Claude lee los artefactos del frontend y genera la lista priorizada de rutas necesarias.

## **02** **Modelo de Base de Datos → YAML Schema**

Un archivo YAML que es documentación, mapa semántico y contexto para la IA — todo a la vez.

## **03** **Supabase + MCP Oficial → Tablas en la Nube**

Claude se conecta a Supabase via MCP y crea todas las tablas directamente. Sin SQL manual.

## **04** **Endpoints en Next.js → Locking + AuditLog**

Rutas REST organizadas por módulo, incluyendo el bloqueo pesimista y la auditoría inmutable.

FASE 1

# EL FRONTEND COMO ESPECIFICACIÓN

Invertir el flujo: los stores y componentes dictan qué endpoints construir

# De acciones de usuario a contratos de API

## Lo que Claude va a leer

- `src/lib/types.ts` — interfaces de dominio
- `src/lib/mock-data.ts` — forma exacta de la data
- `src/store/kanbanStore.ts` — acciones del tablero
- `src/store/projectStore.ts` — acciones de proyectos
- `Specs-MiniJira/backlog.md` — criterios Gherkin

## Lo que Claude va a generar

- Lista priorizada de endpoints (P0 → P2)
- Método HTTP + ruta + payload para cada uno
- Qué store/componente dispara cada llamada
- Casos borde mapeados desde el Gherkin

### Outputs:

`api-contract.md` — legible por el equipo, aprobado por ti.  
`openapi.yaml` — generado en Fase 4 desde el contrato, siempre sincronizado con el código via Swagger UI en `/api/docs`.

## **Actividad 1: Extracción del Contrato**

1

### **Abrir Claude Code en la raíz del repo**

El monorepo tiene frontend/ y backend/. Claude necesita ver ambos. Verificar que src/store/ y Specs-MiniJira/ son accesibles.

2

### **Lanzar el Prompt de Extracción**

Claude lee los artefactos del frontend y genera el listado completo de endpoints. **No escribe código todavía** — solo el contrato.

3

### **Revisar y aprobar api-contract.md**

Verificar que cada acción del store tiene su endpoint correspondiente. Confirmar que los casos borde del Gherkin están contemplados.

4

### **Copiar el contrato a la carpeta del backend**

Este archivo viaja con el proyecto. Será la referencia que Claude use en las fases 3 y 4 para construir sin alucinaciones.



## **Prompt 1: Extracción de Endpoints**



**Actúa como Arquitecto de API. Lee estos artefactos del frontend:**

`@src/lib/types.ts, @src/lib/mock-data.ts, @src/store/kanbanStore.ts,  
@src/store/projectStore.ts y @Specs-MiniJira/backlog.md.`

**Antes de generar nada, usa AskUserQuestion en una sola ronda para preguntarme:  
formato del envelope de respuesta ({ data, error } u otro), convención de  
nombres de rutas, estilo de paginación y formato de errores HTTP.**

**Con mis respuestas, extrae todas las operaciones de datos del frontend y genera  
api-contract.md con método, ruta, payload y response por endpoint, agrupados en  
P0 / P1 / P2.**

**No escribas código. Solo el contrato. Detente y espera mi aprobación.**



## ✓ Checkpoint 1: ¿Tenemos el contrato?



### **api-contract.md generado**

Cada store action tiene su endpoint. Cada criterio Gherkin tiene su ruta de soporte.



### **Endpoints priorizados P0/P1/P2**

Los P0 son los que desbloquean el tablero Kanban. La construcción arranca por ahí.



### **Payloads definidos**

El contrato especifica qué JSON recibe y qué JSON devuelve cada ruta. Sin adivinanzas al conectar el frontend.



### **Locking + AuditLog incluidos**

POST /tickets/[id]/lock, DELETE /tickets/[id]/lock y GET /audit/[ticketId] están en el contrato.



FASE 2

# EL SCHEMA YAML

Un archivo que es documentación, mapa semántico y contexto para la IA — todo a la vez

tables:

tickets:

columns:

id: uuid PK

title: text NOT NULL MAX:255

status: enum(TODO,IN\_PROGRESS,DONE)

priority: enum(HIGH,MEDIUM,LOW)

project\_id: uuid FK→projects.id

created\_by: uuid FK→users.id

archived\_at: timestampz # soft delete

indexes: [status, priority, project\_id]

ticket\_locks:

columns:

ticket\_id: uuid PK FK→tickets.id

locked\_by: uuid FK→users.id

locked\_at: timestampz NOT NULL

expires\_at: timestampz NOT NULL # timeout 15 min

note: "liberar cuando expires\_at < now()"

audit\_log:

columns:

id: uuid PK

ticket\_id: uuid FK→tickets.id

field: text NOT NULL

old\_value: text

SQL es verboso — gasta tokens en sintaxis repetitiva.  
YAML semántico reduce hasta un **60% el consumo de tokens** y permite a Claude entender relaciones entre tablas sin hacer múltiples búsquedas.



Business Development Group S.A.

## El YAML hace 3 trabajos

- ▶ **Documentación:** cualquier dev entiende el modelo
- ▶ **Contexto IA:** Claude lo carga en cada sesión nueva
- ▶ **Blueprint:** Supabase MCP lo usa para crear tablas



## Actividad 2 + Prompt 2: Generar el Schema

### Pasos

1

/clear → nueva sesión limpia en la carpeta del backend.

2

Lanzar el prompt. Claude genera database-schema.yaml con las entidades del Mini Jira.

3

Revisar relaciones FK, campos immutable del AuditLog y el expires\_at del lock.

4

Aprobar. Este archivo viaja con el proyecto para siempre.

Lee @Specs-MiniJira/specs.md y @api-contract.md.

**Genera database-schema.yaml con las entidades del Mini Jira en formato semántico. Para cada tabla incluye: columnas con tipo y restricciones, claves foráneas, índices sugeridos y notas de negocio relevantes (immutable, soft-delete, timeout).**

**No ejecutes ningún comando. Solo genera el archivo YAML.**

## ✓ Checkpoint 2: ¿Tenemos el schema?



### 10 tablas definidas

users, projects, project\_members, tags, tickets, ticket\_assignees, ticket\_tags, comments, audit\_log, ticket\_locks.



### Reglas de negocio en el YAML

audit\_log: immutable: true. ticket\_locks.expires\_at con nota de timeout. Soft deletes marcados con archived\_at.



### FKs y cardinalidades correctas

ticket\_assignees y ticket\_tags como relaciones N:M con PK compuesta. ticket\_locks con PK en ticket\_id (1:1).



### Índices para los filtros del frontend

Columnas status, priority, project\_id indexadas — los mismos filtros que useSearchParams usa en el tablero.

FASE 3

# SUPABASE + MCP OFICIAL

Claude opera la base de datos directamente — sin SQL manual, sin copiar y pegar

# MCP: darle a Claude las manos para operar Supabase

## ✗ Sin MCP

- ▶ Claude genera SQL → tú lo copias → lo pegas en Supabase
- ▶ Cada tabla es una ronda de copiar-pegar
- ▶ Errores de sintaxis que el dev tiene que depurar
- ▶ Claude no sabe si la migración fue exitosa

## ✓ Con MCP Oficial de Supabase

- ▶ Claude lee el YAML y **crea las tablas directamente**
- ▶ Verifica que cada tabla existe antes de continuar
- ▶ Si algo falla, lo corrige en el mismo ciclo
- ▶ Tú solo revisas el resultado en el dashboard de Supabase

**El MCP de Supabase es oficial** — mantenido por el equipo de Supabase. Permite a Claude listar proyectos, ejecutar SQL, crear tablas y leer el esquema de la BD en tiempo real.

# Configurar el MCP de Supabase

## Crear cuenta + Personal Access Token

**01** supabase.com → crear proyecto mini-jira → *Settings* → *Access Tokens* → *Generate*. Copiar el token.

## Registrar el MCP en Claude Code

**02** Agregar en `.claude/settings.json` del proyecto (o en settings globales de Claude Code).

## Verificar con `/mcp`

**03** En la CLI de Claude Code escribir `/mcp`. Debe aparecer **supabase** ✓ **connected**.

### `.claude/settings.json`

```
{
  "mcpServers": {
    "supabase": {
      "command": "npx",
      "args": [
        "-y",
        "@supabase/mcp-server-supabase@latest",
        "--access-token",
        "TU_PERSONAL_ACCESS_TOKEN"
      ]
    }
  }
}
```

El token va en `.env` o en un settings *local* — nunca en el repo.

Crear `.claudeignore` antes de dar acceso a Claude: incluir `.env`, `node_modules/` y `.next/`. Claude no debe ver secretos ni directorios masivos.



## Actividad 3: Supabase + MCP en vivo

1

### Crear cuenta en supabase.com

Nuevo proyecto: `mini-jira`, región más cercana, contraseña de BD anotada. El proyecto tarda ~1 min en inicializarse.

2

### Generar Personal Access Token y configurar MCP

Settings → Access Tokens → Generate. Pegar en `.claude/settings.json` (local, fuera del repo). Verificar con `/mcp` en la CLI.

3

### Lanzar el Prompt de Migración

Claude lee `database-schema.yaml` y crea las 10 tablas en Supabase via MCP. **Humano en el bucle:** revisar cada tabla creada en el dashboard antes de confirmar.

4

### Copiar DATABASE\_URL al .env del backend

Supabase → Settings → Database → Connection string (modo *Session*). Pegar como `DATABASE_URL` en `.env`. El código no cambia si migramos de BD.





## Prompt 3: Migración via MCP

El MCP de Supabase está activo y conectado al proyecto mini-jira.

**Lee @database-schema.yaml y crea todas las tablas en Supabase via MCP. Sigue este orden para respetar las FKs:**

1. users, tags
2. projects, tickets
3. project\_members, ticket\_assignees, ticket\_tags, comments
4. audit\_log, ticket\_locks

**Después de crear cada tabla, verifica que existe antes de continuar con la siguiente. Si una tabla falla, corrígela en el mismo ciclo sin pedirme ayuda. Al terminar, lista todas las tablas creadas.**



# Checkpoint 3: ¿Tenemos la base de datos?



## 10 tablas en Supabase

Verificables en *Table Editor* del dashboard. Sin SQL escrito a mano por el dev.



## Claude verificó cada tabla

El agente consultó la BD después de cada creación. El ciclo fue autónomo — sin copiar-pegar del dev.



## DATABASE\_URL en .env

El string de conexión está aislado en `.env` (en `.claudeignore` y `.gitignore`). El código no sabe qué BD usa.



## MCP configurado para reutilizar

El MCP queda activo en el proyecto. Claude puede consultar el esquema real de la BD en cualquier momento de la sesión 6.



## Trade-off: sin historial de migraciones

Crear tablas via MCP es rápido pero no genera archivos de migración. Para producción real, usar Supabase CLI (`supabase db push`) para tener historial versionable.

FASE 4

# ENDPOINTS EN NEXT.JS

Rutas organizadas por módulo — incluyendo Pessimistic Locking y AuditLog inmutable

## **app/api/ — estructura Next.js App Router**

```
api/
├── projects/
│   ├── route.ts          GET (list) · POST (create)
│   └── [id]/
│       └── route.ts      GET · PATCH · DELETE(soft)
├── tickets/
│   ├── route.ts          GET (filters) · POST (create)
│   └── [id]/
│       ├── route.ts      GET · PATCH
│       ├── lock/
│       │   └── route.ts  POST (acquire) · DELETE (release)
│       ├── assignees/
│       │   └── route.ts  POST · DELETE
│       └── comments/
│           └── route.ts  GET · POST
├── comments/
│   └── [id]/
│       └── route.ts      PATCH · DELETE (soft)
├── tags/
│   └── route.ts          GET (catálogo global)
├── audit/
│   └── [ticketId]/
│       └── route.ts      GET (inmutable, solo lectura)
```

## **Una spec, dos usos: Zod → OpenAPI**



El mismo schema Zod que **valida** el request también **documenta** el endpoint. Sin duplicar definiciones.

- ▶ Librería: `@asteasolutions/zod-to-openapi`
- ▶ Claude anota el schema con `.openapi()` metadata
- ▶ `openapi.yaml` y Swagger UI siempre sincronizados

### **Cada route.ts incluye**

- ▶ Zod schema anotado con metadata OpenAPI
- ▶ Respuesta tipada con envelope `{ data, error }`
- ▶ Try-catch — error interno al log, genérico al cliente
- ▶ `// TODO: verificar JWT — placeholder de auth`

# Los dos endpoints que requieren lógica especial

## **Pessimistic Locking**

**POST /tickets/[id]/lock** — adquirir:

- ▶ Si `ticket_locks` tiene registro activo (`expires_at > now()`) y es de otro usuario → 409 Conflict con datos del bloqueador
- ▶ Si es del mismo usuario → renovar `expires_at`
- ▶ Si no existe o expiró → insertar nuevo lock

**DELETE /tickets/[id]/lock** — liberar: eliminar registro solo si `locked_by = usuario_actual`.

## **AuditLog Inmutable**

El log se escribe dentro del mismo endpoint que modifica el ticket — nunca en un endpoint separado.

- ▶ Cada PATCH `/tickets/[id]` que cambie `status` genera un INSERT en `audit_log`
- ▶ Ambas operaciones van en una **transacción atómica** — si el INSERT de audit falla, el PATCH también se revierte
- ▶ GET `/audit/[ticketId]` es solo lectura — nunca PATCH ni DELETE



## Actividad 4: Construir los Endpoints

1

### **/clear → re-inyectar los artefactos clave**

Nueva sesión con @CLAUDE.md, @api-contract.md, @database-schema.yaml. El contexto limpio garantiza que Claude no alucine rutas que no están en el contrato.

2

### **Fase A — Scaffold + Swagger + Endpoints P0**

Claude crea el proyecto, instala @asteasolutions/zod-to-openapi, genera openapi.yaml desde api-contract.md y levanta Swagger UI en /api/docs. Luego construye los endpoints P0 con sus schemas anotados. Verificar en el browser que /api/docs muestra las rutas.

3

### **/clear → Fase B — Lock + AuditLog (en transacción)**

Nuevo contexto limpio. Claude construye los dos endpoints especiales. Validar con los escenarios borde del Gherkin: timeout de lock, cambio de estado sin lock activo.

4

### **/clear → Fase C — Comments + Assignees + Tags**

Completar los endpoints P1. Al terminar: un endpoint por módulo probado. El mock-data del frontend puede ser reemplazado por llamadas reales.

## Prompts de la Fase 4

### Prompt 4A — Scaffold + Swagger + P0

Lee @api-contract.md, @database-schema.yaml y @CLAUDE.md.

Usa AskUserQuestion: ¿client Supabase (@supabase/supabase-js o directo)?, ¿connection pooling?, ¿TypeScript strict?, ¿envelope de error { data, error }?

Con mis respuestas: **(1)** Crea el proyecto Next.js. **(2)** Crea src/lib/supabase.ts como singleton exportado — nunca instanciar el cliente dentro de un route. **(3)** Instala @asteasolutions/zod-to-openapi. Genera openapi.yaml desde api-contract.md y sirve Swagger UI en /api/docs. **(4)** Construye endpoints P0. Por cada route.ts: schema Zod con .openapi() metadata, envelope { data, error }, try-catch, actualizar openapi.yaml.

### Prompt 4B — Lock + AuditLog

Lee @api-contract.md, @database-schema.yaml y @openapi.yaml.

Construye POST y DELETE /tickets/[id]/lock: lógica de timeout(expires\_at < now()), renovar si mismo usuario, 409 con datos del bloqueador si es otro.

En PATCH /tickets/[id]: cuando status cambie, insertar en audit\_log en **transacción atómica** — si el INSERT falla, el PATCH se revierte. GET /audit/[ticketId] solo lectura.

Por cada endpoint nuevo: schema Zod con .openapi() metadata y actualizar openapi.yaml. Verificar que /api/docs muestra las rutas nuevas.

## ✓ Checkpoint 4: ¿Tenemos el backend?



### Endpoints P0 respondiendo

Projects y Tickets básicos devuelven datos reales de Supabase.  
El tablero Kanban del frontend puede conectarse.



### Lock con los 3 escenarios

Nuevo lock ✓ · Renovar propio lock ✓ · Rechazar lock ajeno con 409 ✓ · Timeout de 15 min via expires\_at ✓.



### AuditLog en transacción

Cambio de estado + registro de auditoría son atómicos. El log es immutable — ningún endpoint permite DELETE en audit\_log.



### Swagger UI en /api/docs

openapi.yaml sincronizado con el código. Cada endpoint tiene schema Zod anotado — documentación que no puede desfasarse del código porque nace del mismo schema.



# Lo que dominamos hoy

## El Frontend como Especificación

`types.ts` + `mock-data.ts` + `stores` → `api-contract.md`. El backend nunca adivina qué construir — el frontend ya lo dice todo.

El YAML es a la vez documentación viva y contexto para la IA en cada nueva sesión.

## MCP = Claude con manos

El MCP oficial de Supabase convierte a Claude en un DBA que crea, verifica y corrige tablas de forma autónoma.

## Locking + Audit en producción

Pessimistic lock con timeout + transacciones atómicas para el audit log — las dos piezas más complejas del Mini Jira, construidas sin inventar lógica fuera del backlog.

## /clear como ritual

Una fase por sesión limpia. El código ya está guardado — Claude solo necesita los artefactos clave, no la historia del chat.

Q & A

**¿PREGUNTAS?**